

KV Cache Hijack

KV Cache Hijack - YichiCS, GitHub.

- Published: date not stated
- Original: <<https://github.com/YichiCS/KV-Cache-Hijack>>
- Preserved from: <https://github.com/YichiCS/KV-Cache-Hijack> (git) on 2026-08-19
- Repository commit: `f9a5c07bddeae9ef4e04582f4ab114f26c9cbec3`
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <<https://github.com/YichiCS/KV-Cache-Hijack>>
- Commit: `f9a5c07bddeae9ef4e04582f4ab114f26c9cbec3`
- Documents preserved: 2

LICENSE

Blob `3cef8a0f5360`, 1057 bytes, at commit `f9a5c07bddea`.

MIT License

Copyright (c) 2026

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN

ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob 1ec5218bdc7c, 5402 bytes, at commit f9a5c07bddea.

HIJACKKV: New Threat in Position-Independent KV Cache Reuse

This repository contains runnable code for the HijackKV attack and for re-evaluating saved attack results with different KV-cache recomputation methods.

Environment

Python is pinned to `>=3.12,<3.13`. Dependencies are managed by `uv` and locked in `uv.lock`.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv sync
```

Experiment Server

The current demo and experiments are run on the following server:

Component	Configuration
GPU	4 x NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition, 97,887 MiB each
NVIDIA driver	580.105.08
CPU	2 x AMD EPYC 9334 32-Core Processor

The bundled `scripts/demo.sh` is configured for this machine by default with `DEVICE=0,1,2,3`. On a different server, adjust `DEVICE` in the script or pass a matching `--device` list to the CLI commands.

Repository Layout

```
.data/
  datasets/
    hotpotqa_divaco_50.json  # 50-sample demo dataset
  assets/
    instruction_question.txt  # System prompt used for attack and evaluation
  scripts/
    demo.sh                  # End-to-end attack and recomputation demo
  src/
    run_attack.py            # Main attack CLI
    run_evaluate.py          # Re-evaluation CLI
  kvcache/
    picache.py               # Position-independent cache utilities
    recomps/                 # vanilla, epic, random, cacheblend methods
  pyproject.toml
  uv.lock
```

Input Dataset

`src/run_attack.py` expects `--dataset` to point to a JSON file. The file can be:

- a list of sample objects, or
- an object with a top-level `result` list.

Each sample used by the attack should contain:

```
{
  "id": 123,
  "context": "...",
  "question": "...",
  "answer": "...",
  "target": "..."
}
```

Samples with an empty `target` are skipped. During optimization, only the first target token is used. The repository includes a small demo dataset:

```
.data/datasets/hotpotqa_divaco_50.json
```

It contains 50 HotPotQA-style samples and can be used directly with `--dataset .`

Run an Attack

```
uv run python src/run_attack.py \  
--dataset path/to/dataset.json \  
--device 0
```

For multiple GPUs, pass a comma-separated device list:

```
uv run python src/run_attack.py \  
--dataset path/to/dataset.json \  
--device 0,1
```

One worker process is launched per listed GPU.

Demo Script

`scripts/demo.sh` runs an end-to-end example on the bundled dataset:

```
bash scripts/demo.sh
```

The script runs attack on `.data/datasets/hotpotqa_divaco_50.json` with `RATIO=0.1`, then re-evaluates the generated attack with `vanilla`, `random`, `epic`, and `cacheblend`.

Edit `DATASET`, `DEVICE`, `RATIO`, or `METHODS` inside the script if your local GPU layout or experiment settings differ.

Common Attack Options

Option	Default	Description
<code>--dataset</code>	required	Input JSON dataset path.
<code>--device</code>	0	CUDA device id or comma-separated ids.
<code>--model</code>	meta-llama/Llama-3.1-8B-Instruct	Hugging Face causal LM.
<code>--gcg_recomp_ratio</code>	0.1	Recompute ratio during GCG optimization.
<code>--gcg_recomp_method</code>	vanilla	One of <code>vanilla</code> , <code>random</code> , <code>epic</code> , <code>cacheblend</code> .
<code>--eval_recomp_ratio</code>	same as <code>--gcg_recomp_ratio</code>	Recompute ratio for the post-attack evaluation pass.
<code>--eval_recomp_method</code>	same as <code>--gcg_recomp_method</code>	Recompute method for the post-attack evaluation pass.
<code>--max_samples</code>	200	Maximum non-empty-target samples to process.
<code>--output_dir</code>	<code>.data/results</code>	Parent directory for attack outputs.

`random` and `cacheblend` use model-internal attention code and currently support Llama and Qwen3 model backends.

Attack Output

Attack runs are written to:

```
.data/results/<dataset>_<gcg_method><gcg_ratio>_<device_tag>_<timestamp>/
```

The directory contains:

```
metrics.json    # Merged attack payload
loss.png        # Loss curves, when available
parts/          # Per-device partial results
```

Each result record keeps the original sample fields and adds outputs such as `benign_answer`, `malicious_answer`, `gcg_loss`, and `gcg_prefix_ids`.

Re-evaluate Results

Use `src/run_evaluate.py` to re-run decoding and metrics for a saved attack payload.

```
uv run python src/run_evaluate.py \
  --dataset .data/results/<experiment_dir>
```

`--dataset` can point to either:

- an experiment directory containing `metrics.json`, or
- a specific JSON result file.

Common Evaluation Options

Option	Default	Description
<code>--dataset</code>	required	Experiment directory or result JSON file.
<code>--model</code>	<code>meta-llama/Llama-3.1-8B-Instruct</code>	Hugging Face causal LM used for re-evaluation.
<code>--device</code>	<code>0</code>	CUDA device id.
<code>--eval_recomp_ratio</code> , <code>--ratio</code>	<code>0.3</code>	Recompute ratio during evaluation.
<code>--eval_recomp_method</code> , <code>--method</code>	<code>vanilla</code>	One of <code>vanilla</code> , <code>random</code> , <code>epic</code> , <code>cacheblend</code> .

Each evaluation run writes a new file under:

```
.data/results/<experiment_dir>/eval/
```

The filename includes the recomputation method, ratio, and an incrementing index:

```
metrics_tau<tau>_<method><ratio>_<index>.json
```

Evaluation output stores:

- `metric` : summary metrics
- `args` : evaluation configuration and source arguments
- `results` : updated per-sample records

Archived from <https://github.com/YichiCS/KV-Cache-Hijack>. Rendered offline from the local Markdown copy.